


ROSEBANK
INTERNATIONAL

# EAPD7111 Part 2

**TechMove Global Logistics Management System**

**Student Number:** ST10498624

**Name:** Gerald Mochati Lehana

**Email:** ST10498624@rcconnect.edu.za

**Module:** EAPD7111 - Enterprise Application Development

**GitHub:** https://github.com/iamgeraldlehana/Edu_ST10498624_EAPD7111_GLMS

## Submission Note

The Windows computer I was using for Part 2 was not available when I prepared this submission. I also tried the school's CloudLabs VM, but no VM appeared in my account, even after completing my profile and using the lab invitation.

I therefore prepared this first upload locally on my Mac. It contains the project source code, database scripts, test results and this explanation.

I plan to upload an updated document after the application is running on Azure. That upload will include screenshots of the application and database, together with the video walkthrough.

## 1. Introduction and Database

For Part 2, I used the plan from Part 1 to build an ASP.NET Core MVC application for TechMove. It keeps clients, contracts and service requests in one project. It uses .NET 8, Entity Framework Core for working with SQL Server, and xUnit for testing. These are the main tools required by the brief.

The main database models are:

| Model | Information stored | Relationship |
|---|---|---|
| Client | Name, contact details and region | One client can have many contracts. |
| Contract | Client, start date, end date, status, service level and agreement path | One contract can have many service requests. |
| ServiceRequest | Contract, description, USD cost, exchange rate, ZAR cost and status | Each request belongs to one contract. |
| ContractNotification | Contract, department, message and date | Records updates for Finance and Compliance. |

GlmsDbContext sets up the links between the models. Costs and exchange rates use decimal values. The database connection settings are in appsettings.json and currently point to Windows LocalDB. These settings can also be changed using an environment variable.

The project includes EF Core migrations, which describe how to create the database tables. It also includes a SQL script at database/migrations/InitialCreate.sql. This script creates the main tables and the department updates table. The script is ready, but I have not yet run it on SQL Server.

The controllers use small service classes to save data and check the rules. For example, ContractService checks the contract details before saving them.

## 2. Design Patterns

I used the three design patterns chosen in Part 1: Abstract Factory, Adapter and Observer [1]. Each pattern has a different job in the project.

### Abstract Factory

Abstract Factory creates a contract and its matching pricing rule. IContractFactory lists the methods that each factory must have. StandardContractFactory creates the Standard contract and pricing rule. PremiumContractFactory creates the Premium contract and pricing rule.

ContractService uses the chosen factory to create the contract. ServiceRequestService uses its pricing rule to calculate the request cost. Standard and Premium currently use the same USD-to-ZAR calculation because the brief does not give a different price for Premium.

### Adapter

CurrencyApiClient gets the exchange rate from the currency API. CurrencyApiAdapter reads the result and puts it into an ExchangeRate object that the application can use. The rest of the application gets the rate through ICurrencyRateService.

This keeps the code for reading the API result in one place. If the API cannot be reached or returns an incorrect result, the adapter returns no rate.

### Observer

When a contract is created or its status changes, ContractEventPublisher tells FinanceObserver and ComplianceObserver. Each observer saves an update for its department.

The updates appear on the Contracts page. They are stored in GLMS and are not sent as emails. Saving the same status again does not add the same update twice.

## 3. Workflow, Files and Currency Conversion

### Contract Workflow

The application includes Draft, Active, On Hold and Expired contract statuses. ServiceRequestWorkflow blocks new service requests for Expired and On Hold contracts. ServiceRequestService checks this rule before saving a request.

Before saving a contract, the service checks that the client exists, the service level and status are valid, and the end date is not before the start date. The forms also show messages when required details are missing.

The Contracts page lets the user filter by status, start date and end date. The code uses LINQ to apply the filters together. A contract must start on or after the chosen start date and end on or before the chosen end date to appear in the results.

### Signed Agreements

Each new contract needs a signed agreement. ContractFileService checks that the filename ends in .pdf and that the file is not empty or larger than 5 MB. Other file extensions are rejected. Each file gets a new name so that it does not replace another file. The file is saved in the server’s uploads/agreements folder.

The contract stores the location of the file. The Contracts page has a link to download the PDF. I still need to test uploading and downloading through the browser with SQL Server connected.

### Currency Conversion

The service request form gets a USD-to-ZAR rate from the Frankfurter API. The code uses HttpClient with async/await to make the call. JavaScript updates the estimated ZAR cost when the user changes the USD amount.

When the user submits the form, the application gets the rate again and calculates the ZAR cost before saving. It saves the USD amount, the rate used and the ZAR amount together. This stops the application from trusting a cost that someone could change in the browser.

The calculation uses decimal values and rounds the result to two decimal places. For example, USD 100 at a rate of 18.25 gives ZAR 1,825.00. If a rate is unavailable, the application shows a message and does not save the request.

## 4. Testing and Current Status

GLMS.Tests is a separate xUnit test project. I ran the following command. All 33 tests passed:

```
dotnet test GLMS.sln --configuration Release
Passed: 33   Failed: 0   Skipped: 0   Total: 33
```

| Test area | Checks included |
|---|---|
| Currency calculation | Conversion, rounding, zero amount and invalid values |
| File validation | PDF extensions accepted; .exe, .docx and empty filenames rejected |
| Contract workflow | Requests allowed for Draft and Active; blocked for Expired and On Hold |
| Abstract Factory | Each factory creates the correct contract and pricing rule |
| Adapter | Reads the API fields; handles a failed call, incorrect JSON and missing or invalid rates |
| Observer | Both departments get updates; saving the same status adds no extra update |

| Test area | Checks included |
| --- | --- |
| Saving requests | The application calculates the saved cost; blocked requests and requests without a rate are not saved |
| Saving and finding contracts | Saves the contract and updates; rejects incorrect dates; applies the filters together |

Some tests use a temporary database held in memory and fixed exchange rates. These tests check the code, but they do not test the SQL Server connection or all the browser steps.

The project also built successfully. The home page loaded in a local check and returned HTTP status 200, which means the request succeeded. A separate check through the application returned a live exchange rate from the API. Saving a service request in SQL Server still needs to be tested.

## References

[1] E. Gamma, R. Helm, R. Johnson and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.